UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Aqila Nike Indriani - 2428240130
Kelas	SI5C
Judul project / topik API	RESTful API Daftar Tugas (Produktivitas) - Topik 18, Tugas 1
Tautan repositori	https://github.com/AqilaNike/tugas1-restful-2428240130
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	14 Oktober 2026

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

API yang diuji adalah RESTful API Daftar Tugas (Topik 18 - Produktivitas) dari Tugas 1, dengan entitas utama tugas (tasks) yang memiliki atribut judul, deskripsi, prioritas, tenggat, dan selesai. API dibangun dengan Express.js dalam satu berkas app.js, menyimpan data pada array di memori, dan seluruh responnya berformat JSON.

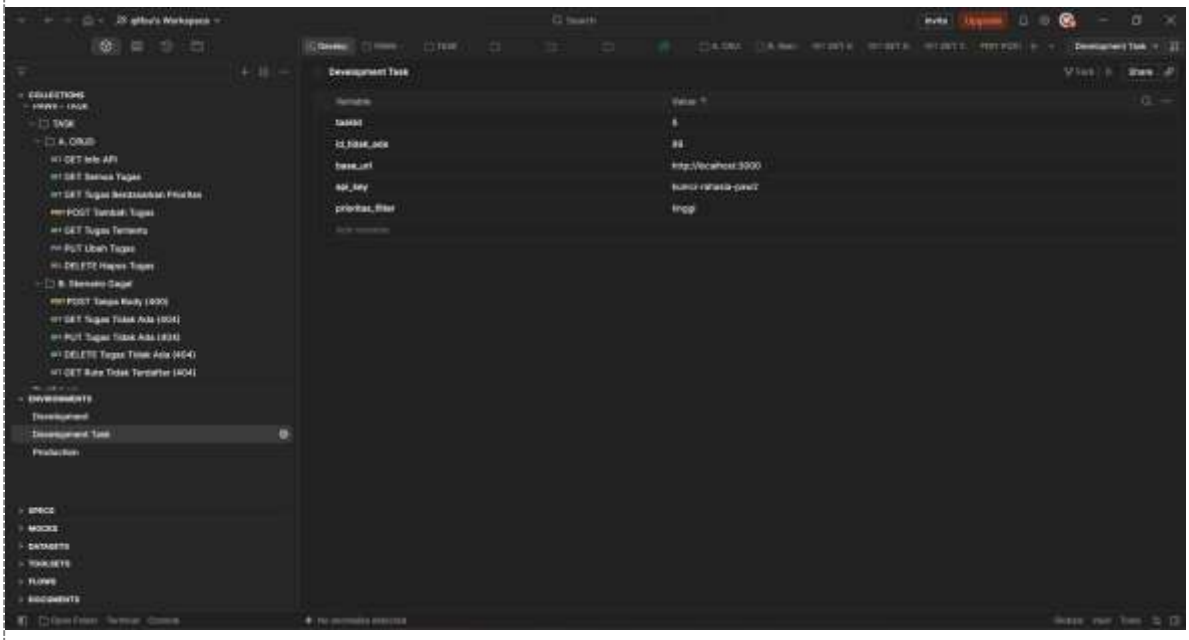
API menyediakan operasi CRUD lengkap beserta filter berdasarkan prioritas melalui query string. Seluruh endpoint bersifat terbuka (tidak mewajibkan API key). Meski begitu, request POST, PUT, dan DELETE pada Postman tetap mengirim header x-api-key yang nilainya diambil dari variabel api_key, sehingga Collection siap dipakai bila API kelak dilindungi kunci.

1.2 Daftar endpoint

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/	Tidak	200	Menampilkan info API (identitas dan daftar endpoint)
2	GET	/tasks	Tidak	200	Menampilkan semua tugas
3	GET	/tasks?prioritas=nilai	Tidak	200	Memfilter tugas berdasarkan prioritas
4	GET	/tasks/:id	Tidak	200	Menampilkan satu tugas (404 bila tidak ada)
5	POST	/tasks	Tidak	201	Menambah tugas baru (400 bila field wajib kosong)
6	PUT	/tasks/:id	Tidak	200	Mengganti seluruh data tugas (400 / 404)
7	DELETE	/tasks/:id	Tidak	200	Menghapus tugas (404 bila tidak ada)

1.3 Variabel Environment

No	Nama variabel	Nilai	Fungsi
1	`base_url`	http://localhost:3000	Alamat server
2	`api_key`	kunci-rahasia-paw2	Kunci rahasia latihan yang dikirim lewat header x-api-key pada request POST, PUT, dan DELETE
3	`taskId`	(kosong, diisi script)	ID dari respons POST /tasks, dipakai GET, PUT, DELETE berikutnya
4	`prioritas_filter`	tinggi	Nilai query string untuk filter prioritas
5	`id_tidak_ada`	99	ID yang tidak terdaftar, dipakai pada skenario 404



Gambar 1. Environment Development Task beserta variabelnya

Penjelasan:

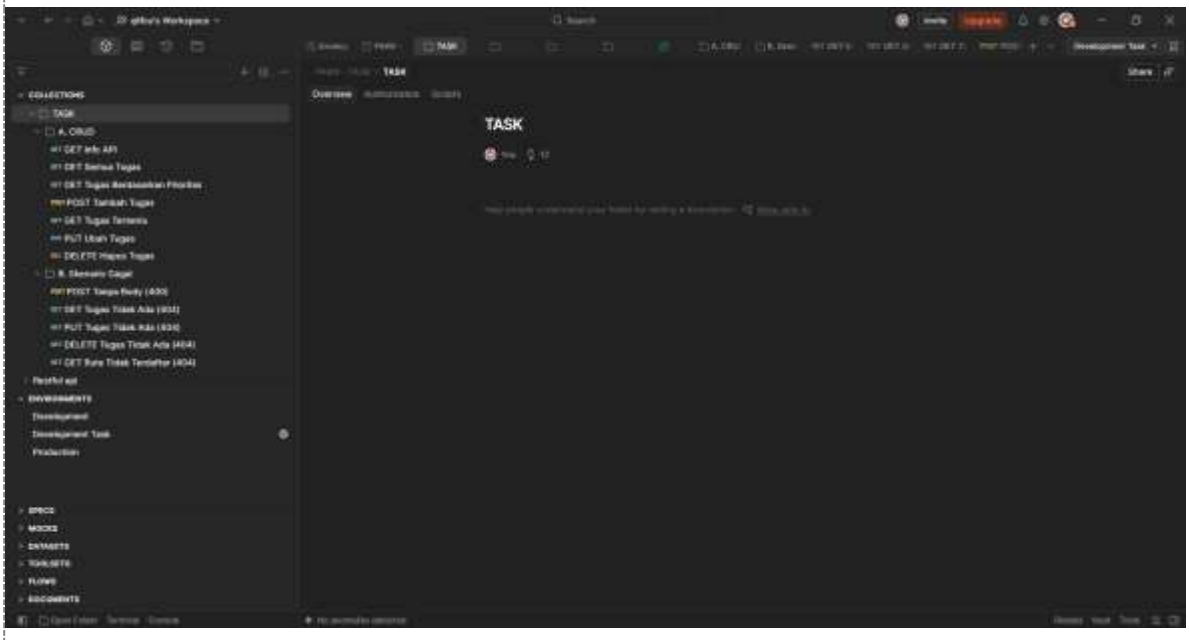
Environment Development Task berisi lima variabel. base_url dan api_key bernilai tetap, prioritas_filter dan id_tidak_ada menjadi nilai masukan untuk request filter dan skenario 404, sedangkan taskId sengaja dikosongkan karena diisi otomatis oleh test script pada request POST Tambah Tugas.

Environment ini harus dipilih pada daftar environment di pojok kanan atas Postman. Tanpa itu, {{base_url}} tidak terbaca dan seluruh request gagal.

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection



Gambar 2. Collection PAWII - TASK beserta seluruh request

Penjelasan:
Saya mengelompokkan 12 request ke dalam Collection PAWII - TASK yang terdiri dari dua folder. Folder A. CRUD berisi 7 request skenario berhasil dengan urutan mengikuti siklus hidup data: GET Info API, GET Semua Tugas, GET Tugas Berdasarkan Prioritas, POST Tambah Tugas, GET Tugas Tertentu, PUT Ubah Tugas, dan DELETE Hapus Tugas. POST sengaja diletakkan sebelum GET Tugas Tertentu, PUT, dan DELETE karena ketiganya memakai taskId yang disimpan oleh POST. Folder B. Skenario Gagal berisi 5 request yang sengaja memicu status 400 dan 404 (POST tanpa body, GET/PUT/DELETE dengan id yang tidak ada, dan rute yang tidak terdaftar). Folder ini diletakkan setelah A. CRUD agar tidak mengganggu data yang dipakai skenario berhasil.

2.2 Tabel request

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Info API	GET	{{base_url}}/	CRUD
2	GET Semua Tugas	GET	{{base_url}}/tasks	CRUD
3	GET Tugas Berdasarkan Prioritas	GET	{{base_url}}/tasks?prioritas={{prioritas_filter}}	CRUD
4	POST Tambah Tugas	POST	{{base_url}}/tasks	CRUD
5	GET Tugas Tertentu	GET	{{base_url}}/tasks/{{taskId}}	CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
6	PUT Ubah Tugas	PUT	{{base_url}}/tasks/{{taskId}}	CRUD
7	DELETE Hapus Tugas	DELETE	{{base_url}}/tasks/{{taskId}}	CRUD
8	POST Tanpa Body (400)	POST	{{base_url}}/tasks	Skenario Gagal
9	GET Tugas Tidak Ada (404)	GET	{{base_url}}/tasks/{{id_tidak_ada}}	Skenario Gagal
10	PUT Tugas Tidak Ada (404)	PUT	{{base_url}}/tasks/{{id_tidak_ada}}	Skenario Gagal
11	DELETE Tugas Tidak Ada (404)	DELETE	{{base_url}}/tasks/{{id_tidak_ada}}	Skenario Gagal
12	GET Rute Tidak Terdaftar (404)	GET	{{base_url}}/rute-tidak-ada	Skenario Gagal

2.3 Header dan body request

Request yang mengubah data (POST, PUT, DELETE) mengirim header x-api-key dari variabel api_key. Request yang mengirim body memakai Content-Type application/json, dan body diisi pada Body > raw > JSON.

No	Nama request	Header	Body
1	GET Info API	-	-
2	GET Semua Tugas	-	-
3	GET Tugas Berdasarkan Prioritas	-	-
4	POST Tambah Tugas	`x-api-key: {{api_key}}` `Content-Type: application/json`	raw JSON, lihat Body A
5	GET Tugas Tertentu	-	-
6	PUT Ubah Tugas	`x-api-key: {{api_key}}` `Content-Type: application/json`	raw JSON, lihat Body B
7	DELETE Hapus Tugas	`x-api-key: {{api_key}}`	-
8	POST Tanpa Body (400)	`x-api-key: {{api_key}}` `Content-Type: application/json`	raw JSON: {}
9	GET Tugas Tidak Ada (404)	-	-
10	PUT Tugas Tidak Ada (404)	`x-api-key: {{api_key}}` `Content-Type: application/json`	raw JSON, sama dengan Body B

No	Nama request	Header	Body
11	DELETE Tugas Tidak Ada (404)	`x-api-key: {{api_key}}`	-
12	GET Rute Tidak Terdaftar (404)	-	-

```
// Body A: POST Tambah Tugas (Body > raw > JSON)
{
  "judul": "Mengerjakan laporan Tugas 2",
  "deskripsi": "Menyelesaikan pengujian API dengan Postman",
  "prioritas": "tinggi",
  "tenggat": "2026-10-20",
  "selesai": false
}
```

```
// Body B: PUT Ubah Tugas (Body > raw > JSON)
{
  "judul": "Mengerjakan laporan Tugas 2 (revisi)",
  "deskripsi": "Pengujian API selesai, tinggal finalisasi laporan",
  "prioritas": "sedang",
  "tenggat": "2026-10-22",
  "selesai": true
}
```

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Info API	Status code 200	Header JSON; nama, nim, dan 6 endpoint pada daftar endpoints
2	GET Semua Tugas	Status code 200	Array tidak kosong; tiap tugas punya judul, prioritas, tenggat
3	GET Tugas Berdasarkan Prioritas	Status code 200	Array; semua prioritas sama dengan prioritas_filter
4	POST Tambah Tugas	Status code 201	status success; data.id bertipe number; data sama dengan yang dikirim; taskId disimpan
5	GET Tugas Tertentu	Status code 200	Objek polos tanpa status/message; id = taskId; judul sesuai
6	PUT Ubah Tugas	Status code 200	status success; id tetap; judul dan selesai berubah
7	DELETE Hapus Tugas	Status code 200	status success; data null; message memuat id
8	POST Tanpa Body (400)	Status code 400	status error; message memuat "wajib"; data null
9	GET Tugas Tidak Ada (404)	Status code 404	status error; message memuat id 99; data null
10	PUT Tugas Tidak Ada (404)	Status code 404	status error; message memuat id 99; data null
11	DELETE Tugas Tidak Ada (404)	Status code 404	status error; message memuat id 99; data null
12	GET Rute Tidak Terdaftar (404)	Status code 404	Header JSON; message "Endpoint tidak ditemukan"; data null

3.2 Cuplikan 1: request GET

```
// Test script request GET: GET Tugas Berdasarkan Prioritas
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons filter berupa array", function () {
    pm.expect(pm.response.json()).to.be.an("array");
});

pm.test("Semua tugas hasil filter berprioritas sesuai variabel prioritas_filter",
function () {
    const prioritas = pm.environment.get("prioritas_filter");
    const sesuai = pm.response.json().every(function (t) {
        return t.prioritas === prioritas;
    });
    pm.expect(sesuai).to.be.true;
});
```

Penjelasan:

Test pertama memastikan server membalas status 200 OK, artinya filter diproses tanpa error. Test kedua memastikan respons berupa array, sebab hasil filter harus berbentuk array walaupun isinya kosong. Test ketiga membaca nilai prioritas_filter dari Environment lalu memeriksa bahwa setiap tugas pada hasil filter benar-benar memiliki prioritas itu, sehingga bila filter pada API salah (misalnya mengembalikan semua tugas) test ini akan gagal.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

```
// Test script request POST: POST Tambah Tugas
pm.test("Status code adalah 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Respons memakai struktur status, message, dan data", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("success");
    pm.expect(json).to.have.property("message");
    pm.expect(json.data).to.have.property("id");
    pm.expect(json.data.id).to.be.a("number");
});

pm.test("Data yang tersimpan sama dengan data yang dikirim", function () {
    const kirim = JSON.parse(pm.request.body.raw);
    const data = pm.response.json().data;
    pm.expect(data.judul).to.eql(kirim.judul);
    pm.expect(data.prioritas).to.eql(kirim.prioritas);
});

// simpan id hanya bila POST berhasil (201)
if (pm.response.code === 201) {
    pm.environment.set("taskId", pm.response.json().data.id);
}
```

Penjelasan:

Setelah POST berhasil (status 201), script membaca data.id dari respons lalu menyimpannya ke Environment dengan pm.environment.set("taskId", ...). Penyimpanan hanya dilakukan bila status 201 agar tidak menyimpan id dari respons yang gagal. Variabel ini dipakai oleh tiga request berikutnya melalui {{taskId}}, yaitu GET Tugas Tertentu, PUT Ubah Tugas, dan DELETE Hapus Tugas.

Bila urutan ditukar sehingga request tersebut dijalankan sebelum POST, {{taskId}} masih kosong dan alamatnya menjadi /tasks/. GET /tasks/ mengembalikan array semua tugas sehingga test "objek polos" gagal, sedangkan PUT dan DELETE ke /tasks/ jatuh ke catch-all dan membalas 404 sehingga test status code gagal.

3.4 Script seluruh request

Berikut script pada tab Scripts > After response untuk seluruh 12 request. Script request 3 dan 4 sama dengan cuplikan pada bagian 3.2 dan 3.3.

Request 1: GET Info API

```
// Test script request 1: GET Info API
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berformat JSON", function () {
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

pm.test("Info API memuat nama, nim, dan 6 endpoint", function () {
    const json = pm.response.json();
    pm.expect(json).to.have.property("nama");
    pm.expect(json).to.have.property("nim");
    pm.expect(json.endpoints).to.be.an("array").with.lengthOf(6);
});
```

Request 2: GET Semua Tugas

```
// Test script request 2: GET Semua Tugas
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berupa array dan tidak kosong", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("array");
    pm.expect(json.length).to.be.above(0);
});

pm.test("Setiap tugas punya judul, prioritas, dan tenggat", function () {
    pm.response.json().forEach(function (t) {
        pm.expect(t).to.have.property("judul");
        pm.expect(t).to.have.property("prioritas");
        pm.expect(t).to.have.property("tenggat");
    });
});
```

Request 3: GET Tugas Berdasarkan Prioritas
Script sama dengan cuplikan pada bagian 3.2.

Request 4: POST Tambah Tugas
Script sama dengan cuplikan pada bagian 3.3.

Request 5: GET Tugas Tertentu
```
// Test script request 5: GET Tugas Tertentu
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berupa objek polos tanpa status dan message", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("object");
    pm.expect(json).to.not.have.property("status");
    pm.expect(json).to.not.have.property("message");
});

pm.test("ID sama dengan taskId", function () {
    const taskId = Number(pm.environment.get("taskId"));
    pm.expect(pm.response.json().id).to.eql(taskId);
});

pm.test("Judul sesuai dengan data yang dikirim saat POST", function () {
    pm.expect(pm.response.json().judul).to.eql("Mengerjakan laporan Tugas 2");
});
```

Request 6: PUT Ubah Tugas
```
// Test script request 6: PUT Ubah Tugas
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons memakai struktur status success dan data", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("success");
    pm.expect(json).to.have.property("message");
    pm.expect(json).to.have.property("data");
});

pm.test("ID tugas tetap sama setelah diubah", function () {
    const taskId = Number(pm.environment.get("taskId"));
    pm.expect(pm.response.json().data.id).to.eql(taskId);
});

pm.test("Judul dan status selesai berubah sesuai data yang dikirim", function () {
    const kirim = JSON.parse(pm.request.body.raw);
    const data = pm.response.json().data;
    pm.expect(data.judul).to.eql(kirim.judul);
    pm.expect(data.selesai).to.eql(true);
});
```

Request 7: DELETE Hapus Tugas

```
// Test script request 7: DELETE Hapus Tugas
pm.test("Status code adalah 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berstatus success dengan data null", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("success");
    pm.expect(json.data).to.be.null;
});

pm.test("Pesan menyebut id tugas yang dihapus", function () {
    const taskId = pm.environment.get("taskId");
    pm.expect(pm.response.json().message).to.include(String(taskId));
});
```

Request 8: POST Tanpa Body (400)

```
// Test script request 8: POST Tanpa Body (400)
pm.test("Status code adalah 400 Bad Request", function () {
    pm.response.to.have.status(400);
});

pm.test("Respons berstatus error dan data null", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("error");
    pm.expect(json.data).to.be.null;
});

pm.test("Pesan error menyebut field wajib", function () {
    pm.expect(pm.response.json().message).to.include("wajib");
});
```

Request 9: GET Tugas Tidak Ada (404)

```
// Test script request 9: GET Tugas Tidak Ada (404)
pm.test("Status code adalah 404 Not Found", function () {
    pm.response.to.have.status(404);
});

pm.test("Respons berstatus error dan data null", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("error");
    pm.expect(json.data).to.be.null;
});

pm.test("Pesan error menyebut id yang tidak ada", function () {
    const id = pm.environment.get("id_tidak_ada");
    pm.expect(pm.response.json().message).to.include(String(id));
});
```

Request 10: PUT Tugas Tidak Ada (404)

```
// Test script request 10: PUT Tugas Tidak Ada (404)
pm.test("Status code adalah 404 Not Found", function () {
    pm.response.to.have.status(404);
});

pm.test("Respons berstatus error dan data null", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("error");
    pm.expect(json.data).to.be.null;
});

pm.test("Pesan error menyebut id yang tidak ada", function () {
    const id = pm.environment.get("id_tidak_ada");
    pm.expect(pm.response.json().message).to.include(String(id));
});
```

Request 11: DELETE Tugas Tidak Ada (404)

```
// Test script request 11: DELETE Tugas Tidak Ada (404)
pm.test("Status code adalah 404 Not Found", function () {
    pm.response.to.have.status(404);
});

pm.test("Respons berstatus error dan data null", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("error");
    pm.expect(json.data).to.be.null;
});

pm.test("Pesan error menyebut id yang tidak ada", function () {
    const id = pm.environment.get("id_tidak_ada");
    pm.expect(pm.response.json().message).to.include(String(id));
});
```

Request 12: GET Rute Tidak Terdaftar (404)

```
// Test script request 12: GET Rute Tidak Terdaftar (404)
pm.test("Status code adalah 404 Not Found", function () {
    pm.response.to.have.status(404);
});

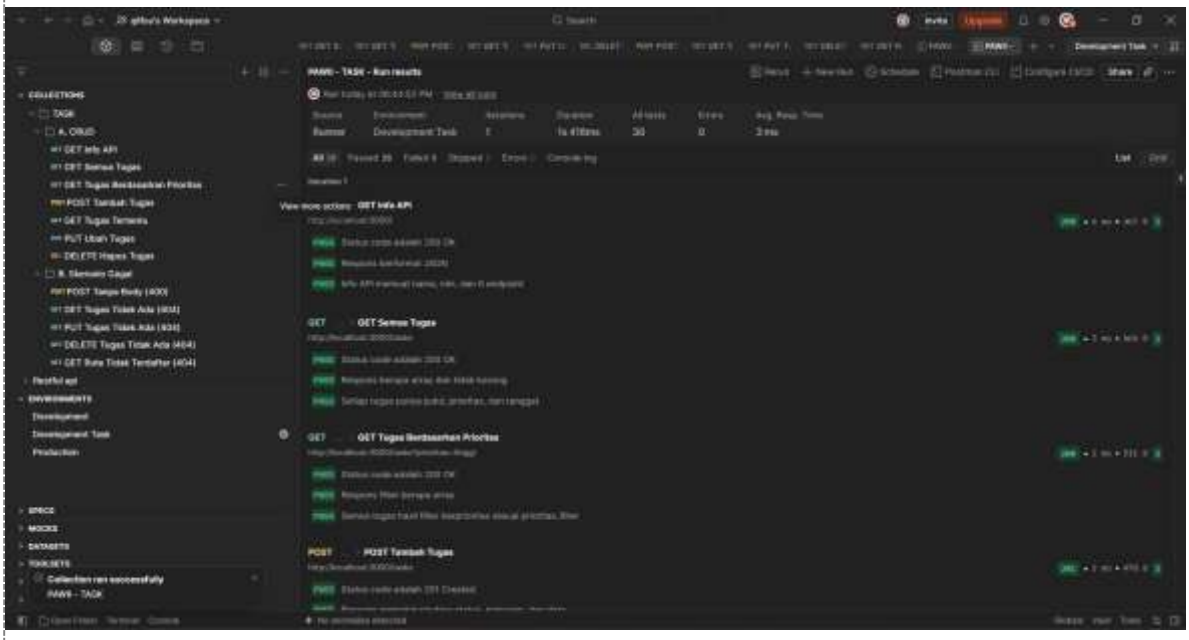
pm.test("Respons berformat JSON", function () {
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

pm.test("Pesan Endpoint tidak ditemukan dan data null", function () {
    const json = pm.response.json();
    pm.expect(json.message).to.eql("Endpoint tidak ditemukan");
    pm.expect(json.data).to.be.null;
});
```

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner



Gambar 3. Ringkasan hasil Collection Runner

Penjelasan:
Collection Runner menjalankan 12 request pada Collection PAWII - TASK dengan Environment Development Task. Seluruh 38 test lulus dan 0 test gagal. Sebelum menjalankan Runner, saya menyalakan ulang server dengan npm run dev agar data kembali ke kondisi awal (tiga tugas), memilih Environment Development Task, lalu menjalankan seluruh Collection.

4.2 Rekap angka hasil

Nama Collection	PAWII - TASK
Environment yang dipakai	Development Task
Jumlah request dijalankan	12
Jumlah test lulus	38
Jumlah test gagal	0
Tanggal dan jam pengujian	14 Oktober 2026

4.3 Temuan selama pengujian

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	GET Info API	AssertionError: expected 'Content-Type' response header to be /application\/json/json but got 'application/json; charset=utf-8'	Pola regex pada pm.response.to.have.header dibaca sebagai teks utuh, sedangkan header asli memuat charset=utf-8	Diganti dengan pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json")

Bab 5 Newman di Terminal (Pengayaan) Tidak Dikerjakan

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/AqilaNike/tugas1-restful-2428240130
Lokasi berkas Collection (.json)	postman/tugas2-collection.json
Lokasi berkas Environment (.json)	postman/tugas2-environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[x] Ya [ ] Tidak
Repositori dapat dibuka dosen	[ ] Ya [ ] Tidak

Bab 7 Refleksi

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Menurut saya assertion yang paling berguna adalah pemeriksaan isi JSON pada request skenario gagal, yaitu status "error", pesan yang memuat id, dan data bernilai null. Status code saja tidak cukup, karena API bisa saja membalas 404 tetapi dengan format pesan yang menyimpang dari aturan Tugas 1, dan hal itu baru ketahuan lewat assertion pada isi JSON.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Pada pengujian manual saya harus mengirim request satu per satu, mengganti id sendiri, lalu membaca hasilnya dengan mata. Dengan Collection Runner, 12 request berjalan berurutan dan hasilnya langsung dirangkum sebagai test lulus atau gagal. Bagian yang paling perlu ketelitian adalah menyimpan taskId dari POST agar bisa dipakai GET, PUT, dan DELETE sesudahnya; saya mengatasinya dengan pm.environment.set pada POST dan memakai {{taskId}} pada alamat request berikutnya. Kesulitan lain muncul pada test header Content-Type yang sempat gagal karena header asli memuat charset=utf-8; saya mengatasinya dengan membandingkan memakai include, bukan mencocokkan teks utuh.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	YA
2	Setiap endpoint API saya ada di Collection	YA
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	YA
4	Setiap request punya assertion status code dan struktur JSON	YA
5	Nama setiap test berupa kalimat yang menyatakan harapan	YA
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	YA
7	Server dinyalakan ulang sebelum Runner dijalankan	YA
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	YA
9	Berkas Environment tidak memuat kunci rahasia sungguhan	YA
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	YA

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen